Duale Hochschule Sachsen
Staatliche Studienakademie Dresden
Studiengang: Informationstechnik - 3IT24-2

Deutsche Telekom AG
Riesaerstraße 5
01129 Dresden

Unterbrechungsfreie SIEM-Migration zu Splunk in einer AWS-Umgebung

Belegarbeit 3. Semester

Friedrich Lidzba
Matrikelnummer: s3005729

Gutachter: Florian Lorch

Tag der Themenübergabe: 12.01.2026
Tag der Einreichung: 27.02.2026

Inhaltsverzeichnis

Inhaltsverzeichnis ..... **II**

Abkürzungsverzeichnis ..... **III**

1 Einleitung **1**

2 Theoretische Grundlagen **3**
2.1 Theoretische Grundlagen von SIEM-Systemen 3
2.2 Architektur und Funktionsweise modernen SIEM-Lösungen 3
2.3 SIEM im Kontext eines Security Operations Centers 4

3 AWS als Betriebsumgebung für Security Information and Event Management (SIEM) **6**
3.1 AWS spezifische Eigenschaften 6
3.2 Relevante AWS Logquellen 7

4 Implementierung von Splunk als SIEM in AWS **8**
4.1 Log-ingestion-Ansätze 8
4.2 Status quo des bestehenden SIEM 9
4.3 Struktur für den Parallelbetrieb 10
4.4 Implementierung mithilfe von Terraform als IaC 11

5 Architektonische Herausforderungen **14**
5.1 Herausforderungen in der bestehenden Architektur 14
5.2 Fehlerbehandlung und Schutzmaßnahmen 14
5.3 Problematik bei Simple Storage Service (S3)-Notifications und Lösung mittels Simple Notification Service (SNS) 15

6 Fazit und Ausblick **16**

Literaturverzeichnis **17**

Anhang **18**

Anhangsverzeichnis **19**

Abbildungsverzeichnis

Tabellenverzeichnis

Abkürzungsverzeichnis

DHSN	Duale Hochschule Sachsen
AWS	Amazon Web Service
SIEM	Security Information and Event Management
SIM	Security Information Management
SEM	Security Event Management
IoT	Internet of Things – Internet der Dinge
SOC	Security Operations Center – Team von Sicherheitsexperten, das IT-Infrastrukturen überwacht
API	Application Programming Interface – Programmierschnittstelle
ES	Splunk Enterprise Security
SPL	Search Processing Language – Abfragesprache zum Durchsuchen und Filtern von Maschinendaten
CIM	Common Information Model
UEBA	User & Entity Behavior Analytics
HEC	HTTP Event Collector
SQS	Simple Queue Service
S3	Simple Storage Service
SNS	Simple Notification Service
IaC	Infrastructure as Code
IAM	Identity and Access Management
KMS	Key Management Service
DLQ	Dead Leatter Queue
DNS	Domain Name System
EC2	Elastic Compute Cloud
Lambda	AWS Lambda
GuardDuty	AWS GuardDuty
CloudTrail	AWS CloudTrail
VPC	Virtual Private Cloud

1 Einleitung

Im Zuge der fortschreitenden Digitalisierung, der zunehmenden Vernetzung von Unternehmensprozessen und der Verbreitung cloudbasierter Architekturen werden moderne IT-Infrastrukturen immer komplexer. Mit Technologien wie dem Internet of Things (IoT) und Multi-Cloud-Landschaften wächst nicht nur das Log-Volumen, sondern auch die Angriffsfläche für Cyberbedrohungen. In diesem Kontext übernehmen SIEM-Systeme eine zentrale Rolle: Sie aggregieren Logs aus diversen Quellen, normalisieren und korrelieren Ereignisse und unterstützen durch forensische Analysen die frühzeitige Erkennung potenzieller Angriffe.

Eine besondere Herausforderung stellt die Sicherheitsüberwachung in Cloud-Umgebungen dar. Cloud-Service-Provider wie Amazon Web Service (AWS) erzeugen durch elastische und skalierbare Ressourcen dynamische, schwer vorhersehbare Logmengen. Der in vielen Unternehmen genutzte Multi-Account-Ansatz von AWS erhöht die Komplexität zusätzlich, da sicherheitsrelevante Informationen aus unterschiedlichen Accounts und Regionen zentral verwaltet und ausgewertet werden müssen. Dienste wie AWS CloudTrail, GuardDuty und Proxy-Logs generieren kontinuierlich sicherheitsrelevante Daten, allerdings in unterschiedlichen Formaten und Strukturen. Ohne ein geeignetes SIEM-System kommt es schnell zu Überwachungslücken, die es Angreifenden ermöglichen, unbemerkt laterale Bewegungen durchzuführen oder sensible Ressourcen zu kompromittieren.

Viele Unternehmen stehen daher vor der Aufgabe, ihre SIEM-Lösungen an moderne Cloud-Architekturen anzupassen oder durch leistungsfähigere Systeme zu ersetzen. Eine solche Migration ist technisch anspruchsvoll, da sie ohne Unterbrechung der Sicherheitsüberwachung stattfinden muss. Datenverluste wären nicht nur aus Compliance-Gründen problematisch, sondern könnten auch dazu führen, dass sicherheitsrelevante Ereignisse nicht erkannt oder nicht mehr forensisch nachvollzogen werden können. Deshalb ist häufig ein paralleler Betrieb von bestehendem und neuem SIEM-System erforderlich, was jedoch eigene technische Herausforderungen mit sich bringt.

Dies betrifft insbesondere Limitationen der AWS-Architektur, wie die Beschränkung bei S3-Benachrichtigungen, die Verarbeitung großer Datenmengen oder die Notwendigkeit einer redundanten und zugleich konsistenten Ereignisweiterleitung. Die Wahl eines geeigneten SIEM-Systems ist daher von entscheidender Bedeutung. Splunk hat sich am Markt als leistungsstarke Plattform zur Verarbeitung großer, heterogener Datenmengen etabliert. Mithilfe der Splunk Enterprise Security App wird die Plattform um essenzielle SIEM-Funktionen wie Ereigniskorrelation, Dashboards, Threat Hunting und maschinelles Lernen erweitert. Zudem bietet Splunk native Integrationsmöglichkeiten für AWS-Services, beispielsweise über den HTTP Event Collector (HEC), Application Programming Interface (API)s oder auf Simple Queue Service (SQS) basierende Pull-Mechanismen.

Diese Eigenschaften machen Splunk zu einem vielversprechenden Kandidaten für die Ablösung eines bestehenden SIEM-Systems in einer AWS-basierten Systemlandschaft. Gleichzeitig ergeben sich bei einer solchen Migration bedeutende architektonische Fragestellungen: Wie können Logquellen zuverlässig angebunden werden? Wie lässt sich ein paralleler Betrieb realisieren, ohne dass Ereignisse dupliziert, verändert oder verloren gehen? Welche Anforderungen entstehen an Rollen- und Berechtigungskonzepte, an Verschlüsselungsmechanismen und an die Automatisierung der Infrastruktur mit Infrastructure as Code? Und wie können typische Herausforderungen wie die Verteilung von S3-Events auf mehrere Systeme technisch gelöst werden?

Diese Überlegungen führen zu der zentralen Forschungsfrage dieser Arbeit:

Forschungsfrage:
Wie kann Splunk als SIEM in einer AWS-Umgebung implementiert werden, sodass ein bestehendes SIEM verlustfrei ersetzt wird, und welche architektonischen Herausforderungen entstehen dabei?

Die vorliegende Arbeit untersucht diese Fragestellung sowohl aus theoretischer als auch aus praktischer Perspektive. Dabei werden die technischen Grundlagen moderner SIEM-Systeme erläutert, die Besonderheiten der AWS-Umgebung analysiert und schließlich ein möglicher Migrationsansatz inklusive typischer Probleme und Lösungsmöglichkeiten detailliert dargestellt.

2 Theoretische Grundlagen

2.1 Theoretische Grundlagen von SIEM-Systemen

Das moderne SIEM-System vereint zwei historischen Konzepten, die zu einem integrierten Gesamtsystem vereint worden sind. Das Security Information Management (SIM) ist dabei eines der beiden Konzepte, dass sich auf die Sammlung und Auswertung von historischen Daten fokussiert, wohingegen das Security Event Management (SEM) Echtzeitüberwachung und Incident Detection bereitstellt. Daraus ist das moderne SIEM-System gewachsen, was sicherheitsrelevante Daten aus unterschiedlichen Quellen zentral sammelt, analysiert und zur Bewertung des Sicherheitszustands eines Unternehmens aufbereitet. Die zentrale Idee besteht darin, sowohl Echtzeit-Analysen als auch historische Untersuchungen in einem einzigen Werkzeug zu vereinen. Damit wird durch ein SIEM ein ganzheitliches Lagebild der IT-Sicherheit ermöglicht, auch wenn die zugrunde liegenden Infrastrukturen zunehmend heterogener und verteilter werden.[1]

Mit der zunehmenden Digitalisierung, der Einführung von Industrie 4.0-Konzepten und der weiter vorangehenden Verbreitung von IoT-Geräten steigt auch die Komplexität der IT-Landschaften erheblich. Dies führt zu einer wachsenden Angriffsfläche sowie einer Vielzahl verteilter Systeme, die isoliert betrachtet nur begrenzte Informationen liefern können. Ein SIEM hingegen ermöglicht es, diese fragmentierten Sichtweisen zu einem konsistenten Gesamtbild zusammenzuführen. Aufgrund von dieser zentralen Rolle ist ein SIEM eng mit einem Security Operations Center (SOC) verknüpft, das mithilfe der Auswertungen sicherheitsrelevante Vorfälle erkennen und behandeln kann. Dabei ist das SIEM ein integraler Bestandteil eines umfassenden Sicherheitsökosystems, anstatt nur ein isoliertes Werkzeug.[2]

Ein SIEM erfüllt eine Reihe von grundlegenden Funktionen, wie die Sammlung von Sicherheitsrelevanter Daten. Diese Daten können entweder per Pull- oder Push-Mechanismus übertragen werden und könne dabei, abhängig von der Systemlandschaft, zentral oder verteilt erhoben werden. Die eingehenden Informationen sind stammen aus zahlreichen Systemen und Formaten und müssen zunächst normalisiert werden, um eine einheitliche Datenbasis zu schaffen. Anschließend führt ein SIEM seine Kernaufgabe aus, die Korrelation und Analyse, um sicherheitsrelevante Muster zu erkennen, woraufhin Alarmierungs- und Reaktionsmechanismen ausgelöst werden. Dazu kommen noch weitere wesentliche Aufgaben, wie das Reporting, Threat Sharing sowie ein umfassendes Log-Management, das Echtzeit- und historische Daten berücksichtigt.[3]

2.2 Architektur und Funktionsweise modernen SIEM-Lösungen

Ein SIEM beginnt mit dem Sammeln der Daten aus einer Vielzahl von Quellen, wie Servern, Netzwerkgeräten, Firewalls oder IDS/IPS-Systemen. Die Erfassung kann dabei entweder über Push-Mechanismen an eine API sein oder über Pull-Mechanismen beispielsweise in Form von Datenbankabfragen oder im Umfeld von AWS über SQS-Queues. Die Herausforderungen finden sich im Verarbeiten der stark heterogenen Log-Formate, weshalb alle Daten einen Normalisierungsprozess durchlaufen. Bei diesem werden die Daten in ein einheitliches Format überführt, woraufhin Parsing- und Analyseprozesse dafür sorgen, dass alle Formate oder Logstrukturen korrekt interpretiert werden können.[4]

[1] Vielberth et al. (Zugriffsjahr: 2026).
[2] Vielberth et al. (Zugriffsjahr: 2026).
[3] Vielberth et al. (Zugriffsjahr: 2026).
[4] Sheeraz et al. (Zugriffsjahr: 2026).

Nach der Normalisierung beginnt die eigentliche sicherheitsrelevante Auswertung der Daten durch die Ereigniskorrelation. Diese fasst einzelne Logeinträge zu größeren Zusammenhängen zusammen, um Muster zu erkennen, die auf Angriffe oder Fehlkonfigurationen hinweisen können. Dazu durchläuft ein SIEM verschiedene Stufen, darunter Event-Filtering, Aggregation, De-Duplication, Event-Masking und die Root-Cause-Analyse. Viele SIEM-Systeme arbeiten dabei regelbasiert, indem Schwellenwerte, Muster oder logische Bedingungen definiert werden. Andere Systeme nutzen Machine Learning, um solche Ereignisse zu erkennen. Wenn dann ein solches Ereignis erkannt wird, erzeugt das SIEM in Echtzeit einen Alarm, der anschließend an die Benutzeroberfläche weitergeleitet wird.[5]

Mithilfe der grafischen Benutzeroberfläche eines SIEM lassen sich komplexe sicherheitsrelevante Informationen verständlich und einfacher darstellen. Durch Visualisierungen wie Diagrammen, Graphen oder Pop-up-Hinweisen erleichtert sich die Identifikation von kritischen Ereignissen. Ergänzend bietet ein SIEM umfangreiche Reporting-Funktionen. Typische Berichte umfassen beispielsweise Authentifizierungsaktivitäten, Zugriffsversuche, Änderungen an Benutzergruppen oder sicherheitsrelevante Vorfälle. Sie leisten einen wichtigen Beitrag zu Compliance-Vorgaben und Audit-Anforderungen.[6]

Durch das Speichern der großen Datenmengen, die ein SIEM-System in Echtzeit verarbeiten muss, setzen modern Architekturen auf skalierbare Datenbanken oder Big-Data-Technologien. Normalisierte Daten und Alerts werden fortlaufen gespeichert, während ältere Datensätze automatisiert in Archivsysteme ausgelagert werden. Für die Archivierung eignen sich Medien wie Cloud-Speicher, Tape-Systeme oder SSDs. Durch die Nutzung von Cloud-Speicher lässt sich das ganze System auch in der Archivierung einfach skalieren.

Darüber hinaus unterstützen SIEM-Lösungen forensische Analysen, indem sie sowohl Host- als auch Netzwerkdaten auswerten können. Maschinelles lernen kann genutzt werden, um Auffälligkeiten schneller zu identifizieren.

2.3 SIEM im Kontext eines Security Operations Centers

Splunk als Beispiel eines SIEM-Systems Splunk ist im Allgemeinen eine Plattform zur Verarbeitung und Analyse maschinell erzeugter Daten in Echtzeit. Diese Plattform wird durch die Erweiterung um die Splunk Enterprise Security (ES)App, zu einem vollwertigen SIEM-System. Die ES-App baut dabei auf der zugrunde liegenden Splunk-Enterprise-Plattform auf und ergänzt sie um ein paar sicherheitsspezifische Funktionen, wie Korrelation, Dashboards und Incident-Management.[7]

Splunk basiert auf einer modularen Architektur, bei der die Splunk Forwarder als Datensammler fungieren und Logs aus verschiedenen Datenquellen wie Firewalls, Endgeräten oder Cloud-Systemen übertragen. Dabei unterscheidet Splunk zwei Arten von Forwardern. Zum einen die Universal Forwarder, die die rohen Daten ohne eine Priorisierung weiterleiten und den Heavy Forwardern, die nur die analysierten Events an die Indexer weiterleiten, was wiederum Ressourcen intensiver ist.[8]

Die Indexer übernehmen Parsing, Feldextraktion, Indexierung und das Speichern der Daten. Dabei nutzen größere Umgebungen skalierbare Indexer-Cluster hierfür. Der Search Head ermög-

[5] Sheeraz et al. (Zugriffsjahr: 2026).
[6] Sheeraz et al. (Zugriffsjahr: 2026).
[7] Ojuko; Clouidan; Splunk; Basatwar (Zugriffsjahr: 2026).
[8] Clouidan (Zugriffsjahr: 2026).

licht Suchabfragen über Search Processing Language (SPL) und bietet Funktionen wie Dashboards, Reporting und Threat Hunting. Die Haupt Komponente bildet die Big-Data-Plattform für Speicherung, Suchoptimierung und Visualisierung.[9]

Die ES-App stellt eine Sicherheitsoberfläche bereit, de Correlation Searches, Notable Events und sicherheitsrelevante Dashboards umfasst. Dabei sorgen verschiedene Add-Ons dafür, dass unterschiedliche Datenformate korrekt geparst und in ein einheitliches Datenmodell überführt werden. Dieses Common Information Model (CIM) bildet die Grundlage für die Korrelation. Durch die regelbasierte oder SPL-basierte können komplexe Angriffe über verschiedene Quellen hinweg erkannt werden. Dashboards zeigen Bedrohungsaktivitäten und Risikoindikatoren an. Ergänzend bietet Splunk Machine-Learning-Funktionen und User & Entity Behavior Analytics (UEBA) um Anomalien zu erkennen. Somit bietet Splunk viele Möglichkeiten, um auch in großen und komplexen Cloud Umgebungen viele Gefahren und Angriffe zu erkennen, um damit schnell reagieren zu können.[10]

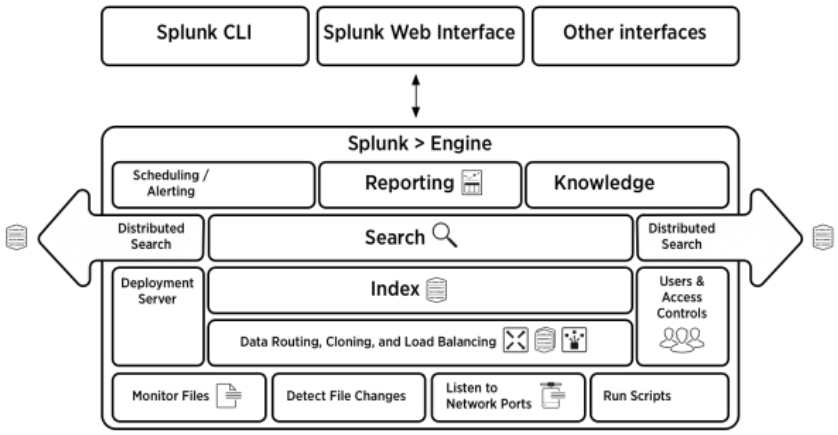


Abbildung 1: Splunk Architektur Diagramm

Quelle: [11]

[9] Clouidan (Zugriffsjahr: 2026).
[10] Ojuko; Basatwar (Zugriffsjahr: 2026).

3 AWS als Betriebsumgebung für SIEM

In einer Cloud-Umgebung werden Ressourcen und Infrastrukturen auf verschiedene Accounts und Regionen verteilt, wodurch ein unkomplizierter Überblick über potenzielle Gefahren erschwert wird. Daher gewinnt der Einsatz eines SIEM-Systems zunehmend an Bedeutung. Das SIEM ist in der Lage, korrelierende API-Ereignisse, Netzwerkaktivitäten, Domain Name System (DNS)-Anfragen und Änderungen im Identitäts- und Zugriffsmanagement (Identity and Access Management (IAM)) zu erkennen und – falls erforderlich – die Benutzer darüber zu informieren bzw. zu alarmieren.

3.1 AWS spezifische Eigenschaften

Das AWS Well-Architected Framework beschreibt bewährte Methoden für den Aufbau einer leistungsfähigen AWS-Umgebung. Eine der sechs zentralen Säulen ist dabei die Sicherheit, deren Ziel es ist, die AWS-Cloud vor unbefugten Zugriffen zu schützen und derartige Aktivitäten frühzeitig zu erkennen. Dazu gehört auch, standardisierte Verfahren zur Sammlung und Analyse von Logs und Metriken zu etablieren, um Sicherheitsereignisse gezielt untersuchen zu können.[12]

AWS empfiehlt hierfür den Einsatz eines SIEM-Systems oder vergleichbarer Lösungen, um Sicherheitsvorfälle schneller analysieren und darauf reagieren zu können. Zudem wird eine unternehmensweite Strategie für das Sammeln und Auswerten von Daten empfohlen, um systemweite Erkenntnisse zu gewinnen und nicht nur einzelne Bereiche isoliert zu betrachten.[13]

Das Logging für ein SIEM-System stellt in einer AWS-Umgebung eine besondere Herausforderung dar. Ein wesentlicher Grund dafür ist die Elastizität der AWS-Ressourcen, wodurch unvorhersehbare Mengen an Logs anfallen können. Das automatische Skalieren von Instanzen, das zu häufigen An- und Abschaltungen führt, sorgt ebenso dafür, dass innerhalb kurzer Zeiträume viele Logs generiert werden.[14]

Auch serverlose Funktionen wie AWS Lambda (Lambda) erstellen Logeinträge, obwohl sie oft nur für wenige Millisekunden ausgeführt werden und dabei ein großes Volumen an Logs verursachen können. Zusätzlich sind AWS-Infrastrukturen häufig auf viele einzelne Accounts verteilt, was bedeutet, dass die Logdaten zunächst an eine zentrale Sammelstelle weitergeleitet werden müssen. Diese zentrale Analyseplattform muss zudem Logs aus unterschiedlichen Regionen aufnehmen können. Ohne eine solche Logaggregation könnten sicherheitsrelevante Ereignisse unentdeckt bleiben, da Bedrohungen sich über zahlreiche Accounts verteilen können. Die Korrelation der Logdaten wird durch die Multi-Account- und Multi-Region-Architektur erheblich komplexer, weshalb ein leistungsstarkes SIEM-System unverzichtbar wird.[15]

Entsprechend dem Shared Responsibility Model sichert AWS die serverseitige Infrastruktur ab, während die Verantwortung für Logging, Konfiguration und Monitoring beim Kunden liegt. Die meisten Sicherheitslücken entstehen daher im Verantwortungsbereich der Kunden. Insbesondere wenn Ressourcen falsch konfiguriert sind oder Logging fehlt, können blinde Flecken für das SIEM-System entstehen, beispielsweise durch deaktiviertes AWS CloudTrail (CloudTrail) oder

[12] (Zugriffsjahr: 2026).
[13] (Zugriffsjahr: 2026).
[14] Anthony et al. (Zugriffsjahr: 2026).
[15] Anthony et al. (Zugriffsjahr: 2026).

abgeschaltete Virtual Private Cloud (VPC) Flow Logs.[16]

Die Fähigkeit des SIEM-Systems, Logs zu normalisieren, ist im AWS-Kontext zwingend erforderlich, um unterschiedliche Logdaten korrelieren zu können. Denn die verschiedenen AWS-Services produzieren Logs in stark unterschiedlichen Schemata und Formaten. Während CloudTrail beispielsweise Logs als JSON-Einträge generiert, werden VPC Flow Logs als Flat-File Netzwerk-Metadaten festgehalten.[17]

3.2 Relevante AWS Logquellen

Im Rahmen dieser Arbeit liegt der Schwerpunkt auf drei zentralen AWS-Logquellen, die in Splunk integriert werden sollen. Zu diesen Quellen zählt unter anderem CloudTrail. Mit CloudTrail können sämtliche API-Aufrufe innerhalb der gesamten AWS-Umgebung erfasst und gespeichert werden. Dabei werden nicht nur API-Calls von Services wie IAM, Elastic Compute Cloud (EC2), S3 und Lambda aufgezeichnet, sondern auch das Multi-Account-Logging unterstützt, was CloudTrail zu einem essenziellen Bestandteil der Sicherheitsarchitektur macht. Zudem bildet CloudTrail die Grundlage für verschiedene Sicherheitsdienste, wie beispielsweise AWS GuardDuty (GuardDuty).[18]

GuardDuty ist ein Dienst, der Logdaten mittels Machine Learning analysiert. Neben CloudTrail-Logs werden hierfür auch VPC Flow Logs und DNS Query Logs ausgewertet. Auf Basis dieser Daten kann GuardDuty Bedrohungsmuster wie Credential Exfiltration oder ungewöhnliche Nutzung von API-Aufrufen erkennen. Zudem ermöglicht GuardDuty die Identifikation von anomalen Geolokationen anhand von IP-Adressen. Die von GuardDuty generierten Findings können wiederum in das SIEM-System eingebunden werden, um weitergehende Analysen durchzuführen und dadurch die Sicherheit der gesamten Systemlandschaft weiter zu erhöhen.[19]

Abschließend werden in dieser Arbeit auch die Proxylogs in die Implementierung einbezogen. Diese Logs liefern Informationen über den Netzwerkverkehr sowie detaillierte Angaben zu den empfangenen Paketen und den durchgeführten Aktionen entsprechend den festgelegten Regeln. Proxylogs enthalten grundlegende Informationen wie den Namen des Proxys, einen Zeitstempel und die AWS Account ID. Darüber hinaus umfassen sie URL-bezogene Komponenten wie die Zieldomain, den Pfad und das genutzte URL-Schema. Zusätzlich geben die Logs Auskunft über die verwendete HTTP-Methode und den HTTP-Statuscode.[20]

Im Gegensatz zu den Proxylogs werden die VPC Flow Logs in dieser Arbeit nicht näher betrachtet. Während Proxylogs detaillierte Informationen zu den übertragenen Inhalten liefern, konzentrieren sich VPC Flow Logs auf die Metadaten der Netzwerkverbindungen. Diese enthalten Angaben über den IP-Verkehr, wie Quell- und Zieladresse, Ports sowie das verwendete Protokoll. Die Flow Logs sind in Textformaten abgelegt, die keine strukturierten JSON-Felder enthalten, und dienen in erster Linie dazu, ungewöhnlichen Netzwerkverkehr oder Hinweise auf Lateral Movement zu identifizieren.[21]

[16] Anthony et al. (Zugriffsjahr: 2026).
[17] Anthony et al. (Zugriffsjahr: 2026).
[18] Anthony et al. (Zugriffsjahr: 2026).
[19] Anthony et al. (Zugriffsjahr: 2026).
[20] (Zugriffsjahr: 2026).
[21] Anthony et al. (Zugriffsjahr: 2026).

4 Implementierung von Splunk als SIEM in AWS

4.1 Log-ingestion-Ansätze

Um die Verarbeitung von Protokolldaten durch Splunk zu ermöglichen, müssen diese zunächst in das Splunk-System überführt werden. Grundsätzlich lassen sich zwei Ansätze zur Log-Ingestion unterscheiden. Der erste Ansatz beruht auf dem „Push“-Prinzip, bei dem die Protokolldaten aktiv an Splunk übertragen werden. Dies erfolgt typischerweise durch das direkte Senden der Daten an einen HTTP Event Collector (HEC)- oder API-Endpunkt, beispielsweise unter Verwendung einer Lambda-Funktion. Voraussetzung für diese Methode ist, dass die Splunk-API dauerhaft erreichbar ist und über ausreichende Kapazitäten zur Bearbeitung eingehender Anfragen verfügt. Ein wesentlicher Vorteil dieses Übertragungsweges liegt in der nahezu Echtzeit-Verfügbarkeit der Daten, wodurch auch die anschließende Analyse nahezu verzögerungsfrei erfolgen kann.

Im Gegensatz dazu basiert der „Pull“-Ansatz darauf, dass die zu verarbeitenden Protokolldaten zunächst in einer SQS-Queue zwischengespeichert werden. Splunk holt diese Daten anschließend in festen Intervallen selbständig aus der Queue ab. Dieser Ansatz bietet die Möglichkeit, Protokolldaten auch nach Ausfällen oder Verzögerungen vollständig zu verarbeiten, da sie zwischenzeitlich persistiert werden. Zudem ist für diese Methode die Einrichtung eines öffentlichen Endpunkts nicht erforderlich, was das Sicherheitsrisiko reduziert. Darüber hinaus kann eine Überlastung vermieden werden, da der Splunk-Indexer eigenständig steuern kann, welches Datenvolumen er zu einem bestimmten Zeitpunkt verarbeitet. Die Queue verhindert so den Rückstau am HEC-Endpunkt, falls es zu einer kurzfristig erhöhten Event-Last kommt.

Kriterium	**Push-Ansatz (z. B. HEC/API)**	**Pull-Ansatz (z. B. SQS → Splunk)**
Funktionsprinzip	Daten werden aktiv an einen Splunk-Endpoint gesendet (HTTP Event Collector oder API).	Logs werden zuerst in einer Queue gespeichert. Splunk ruft sie selbstständig ab.
Latenz	Sehr gering, nahezu Echtzeit.	Mittel – abhängig vom Polling-Intervall.
Ausfallsicherheit	Niedrig: Bei API-Fehlern oder HEC-Nichtverfügbarkeit besteht Risiko von Event-Verlust.	Hoch: Persistente Queue speichert Events zuverlässig bis zur Verarbeitung.
Skalierbarkeit	Splunk muss eingehende Last direkt verarbeiten.	Splunk kann Verarbeitungsrate selbst steuern → kein Überlastungsrisiko.
Komplexität der Infrastruktur	API-Gateway, Firewall-Regeln, IAM-Policies für öffentlichen Endpoint notwendig.	Einfachere Architektur, da kein öffentlicher Endpoint nötig ist.
Sicherheitsniveau	Public Endpoint → höheres Angriffsrisiko.	Keine öffentlichen Endpoints → geringere Angriffsfläche.
Kosten	Geringe Infrastrukturkosten, aber potenziell teure Splunk-Overheads bei Lastspitzen.	Zusätzliche Kosten durch SQS, dafür stabilere Lastverteilung.

Tabelle 1: Vergleich Push- vs. Pull-Ansatz zur Log-Integration in Splunk

Aus diesen Gründen wird für die vorliegende Arbeit der Pull-basierte Ansatz bevorzugt. Auch weil über diesen Ansatz die Infrastruktur erheblich leichter und übersichtlicher bleibt. Für den Push-basierten Ansatz müsste der API-Endpunkt zuerst geschaffen werden, was die Konfiguration einer Firewall und dem erstellen eines API-Gateway mit sich bringt. Hinzu kommt das Risiko von einem Event-Verlust bei möglichen API-Fehlern.

4.2 Status quo des bestehenden SIEM

Um die Vorgehensweise bei der Implementierung von Splunk als SIEM nachvollziehen zu können, ist es zunächst erforderlich, die grundlegende Funktionsweise des bestehenden SIEM-Systems zu analysieren. Das derzeit eingesetzte SIEM nutzt einen zentralen S3-Bucket, in dem die relevanten Protokolldaten aus einer Vielzahl von Accounts innerhalb der Organisation aggregiert werden. Diese Protokolldaten umfassen verschiedene Dienste, beispielsweise AWS GuardDuty, AWS CloudTrail sowie Proxy-Logs. Für jeden dieser Dienste ist im zentralen Bucket eine spezifische Bucket-Notification konfiguriert, die ausgelöst wird, sobald ein neues Objekt in das jeweilige Verzeichnis des Dienstes geschrieben wird. Die Notification wird anschließend an eine SQS-Queue übermittelt, aus der das bestehende SIEM-System gemäß dem Pull-Prinzip die Benachrichtigung abruft. Auf Basis der übermittelten Informationen zum Dateipfad ist anschließend eine gezielte Analyse der jeweiligen Logdatei möglich.

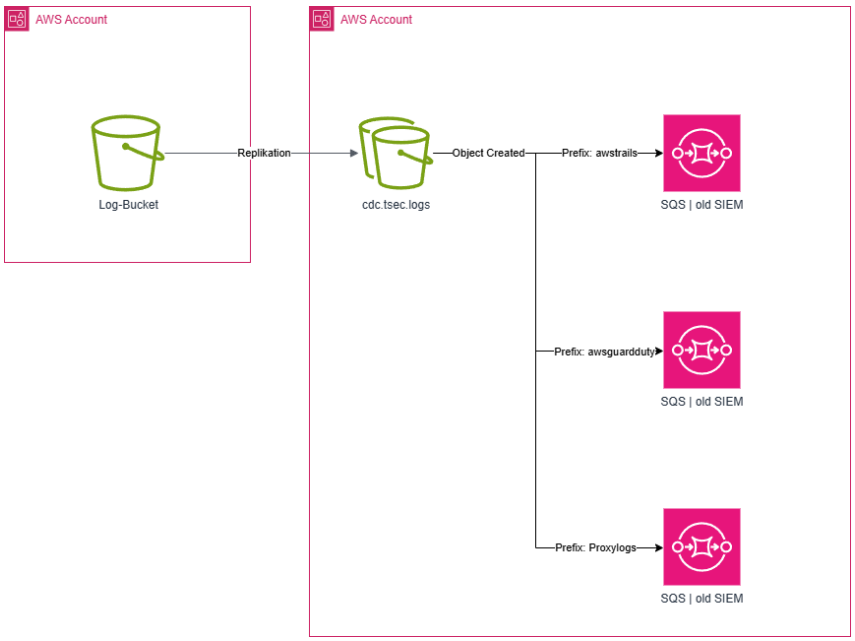


Abbildung 2: Architektur Diagramm Status Quo

Da das bisher eingesetzte SIEM ist eine Eigenentwicklung der Deutschen Telekom IT, die jedoch aufgrund von Sparmaßnahmen nicht mehr weiter gepflegt werden konnte. Durch die abnehmende Zahl an Entwicklern verzögerte sich die Anbindung neuer Logquellen zunehmend, was die Notwendigkeit für ein neues Produkt mit sich brachte. Die Entscheidung für ein externes Produkt fiel schließlich durch das Security-Team auf Splunk.

4.3 Struktur für den Parallelbetrieb

Um die Integration von Splunk als neues SIEM-System unterbrechungsfrei und ohne Datenverlust zu realisieren, ist zumindest für einen begrenzten Zeitraum ein Parallelbetrieb erforderlich. Dadurch kann Splunk in einer kontrollierten Umgebung implementiert und konfiguriert werden, während das bestehende SIEM weiterhin in Betrieb bleibt.

Im Rahmen des Parallelbetriebs werden die Bucket-Notifications nicht mehr direkt an die SQS-Queue des vorhandenen SIEM-Systems weitergeleitet, sondern zunächst an eine SNS-Topic übertragen. Diese SNS-Topic leitet die Nachrichten im Originalformat weiter, um zu gewährleisten, dass Änderungen an der Konfiguration nicht zu Inkompatibilitäten beider SIEM-Systeme führen. Hintergrund ist, dass SNS unter bestimmten Umständen das Nachrichtenformat modifiziert, was die Verarbeitung erschweren kann. Das Modifizieren ist ein Feature von AWS-SNS, wodurch der Payload für alle Protokolle geieignet ist. Jedoch erwarten sowohl das bestehende, als auch das neue SIEM-System eine genaue Struktur der Benachrichtigungen.

Mithilfe der SQS-Topic ist es möglich, identische Benachrichtigungen sowohl an die SQS-Queue des bestehenden SIEM-Systems als auch parallel an die entsprechende Queue für Splunk zu versenden.

Diese Konfiguration ermöglicht es beiden Systemen, simultan und unabhängig voneinander auf die eingehenden Benachrichtigungen zuzugreifen und die jeweiligen Logdateien aus dem S3-Bucket zu beziehen.

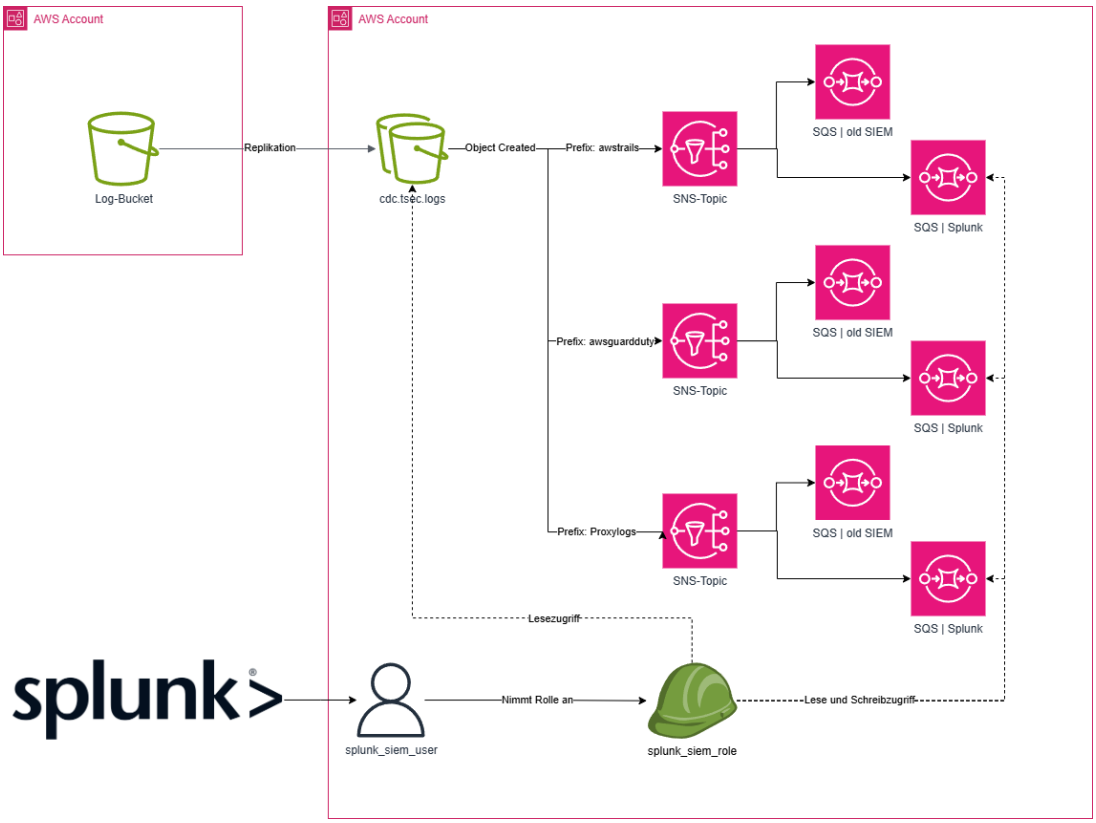


Abbildung 3: Architektur Diagramm Parallelbetrieb

Im Gegensatz zum Fan-out-Prinzip eines SNS besteht auch die Möglichkeit, Benachrichtigungen durch den Einsatz einer Lambda-Funktion zu vervielfältigen und weiterzugeben. Dieser Ansatz bringt jedoch einen deutlich höheren Implementierungsaufwand mit sich, da das Schreiben, Testen und Bereitstellen des Codes erforderlich ist. Hinzu kommen zusätzliche Anforderungen an IAM-Richtlinien und ein erweitertes Error-Handling. Darüber hinaus führt die zusätzliche Lambda-Schicht zu einer erhöhten Latenz im Datenfluss und mindert die Transparenz des Systems gegenüber der Lösung mit SNS-Topics spürbar.

4.4 Implementierung mithilfe von Terraform als IaC

Für die Implementierung kommt neben Terraform auch CloudFormation als Infrastructure-as-Code-Lösung in Betracht. Die bestehende Infrastruktur wurde jedoch bereits vollständig mit Terraform realisiert, da diese Sprache in unterschiedlichen Cloud-Umgebungen einsetzbar ist. Zusätzlich bietet Terraform die Möglichkeit, wiederverwendbare Module zu definieren und flexibel einzusetzen. Ein weiterer Vorteil von Terraform ist die Funktion, bestehende Infrastrukturen zu importieren – eine Option, die CloudFormation nicht zur Verfügung stellt. Ebenso fehlt in CloudFormation eine Funktion zur Planung von Änderungen vor deren Ausführung, wie sie Terraform durch das Plan-Feature bereitstellt.

Damit Splunk über das Splunk-Add-on auf AWS-Ressourcen zugreifen kann, ist ein dedizierter AWS-IAM-Benutzer erforderlich. Dieser ist dazu berechtigt, eine speziell für Splunk konfigurierte IAM-Rolle zu übernehmen. Über diese Rolle erhält Splunk die notwendigen Zugriffsrechte auf die neu erstellten SQS-Queues, um Benachrichtigungen zu empfangen und zu verarbeiten. Hierzu zählen insbesondere Berechtigungen zum Ändern der Sichtbarkeit von Nachrichten

in der Queue, zum Löschen von Nachrichten, zum Auslesen von Attributen einer SQS-Queue sowie – vorrangig – zum Empfangen von Nachrichten.

Darüber hinaus beinhaltet die IAM-Rolle Zugriffsrechte auf den Key Management Service (KMS)-Schlüssel des verwendeten S3-Buckets sowie auf einen für SQS-Queues und SQS-Topics neu angelegten Schlüssel. Dadurch kann Splunk sowohl Nachrichten entschlüsseln als auch neue Daten verschlüsseln, um einen sicheren und datenschutzkonformen Zugriff zu gewährleisten.

Ergänzend ist es erforderlich, dass die Rolle auch auf den entsprechenden S3-Bucket, in dem die Logdateien gespeichert sind, zugreifen kann. Nur so kann Splunk die gespeicherten Protokolldaten zur weiteren Analyse verarbeiten.

Die genaue Konfiguration der IAM-Rolle sowie der zugehörigen Policies für Splunk ist im Anhang A dargestellt.

Die bestehende AWS-Konfiguration des aktuellen SIEMs wurde ausschließlich über die Management-Konsole erstellt. Deshalb ist es von Vorteil, die zu überarbeitende Infrastruktur zunächst mittels Terraform als Infrastructure as Code (IaC)-Lösung zu importieren. Somit lassen sich Veränderungen an der Infrastruktur schneller feststellen und können leichter nachvollzogen werden.

Terraform bietet hierfür sogenannte Import-Blöcke, mit deren Hilfe beim erstmaligen Ausführen die Konfigurationen des S3-Buckets sowie des KMS-Schlüssels in das Terraform-Statefile übernommen werden können. Dadurch lassen sich die bestehenden Ressourcen in Codeform abbilden und im weiteren Verlauf gezielt anpassen.

Mit Hilfe des so generierten Terraform-Codes können anschließend sowohl die Bucket-Policy als auch die bestehende KMS-Policy so erweitert werden, dass die Splunk-Rolle künftig auch auf den verschlüsselten Bucket zugreifen kann.

Auch die bestehenden Bucket-Notifications können auf diese Weise modifiziert werden. Um eine übersichtliche und strukturierte Codierung sicherzustellen, wird für die Hinzufügung einzelner SNS-Topics die Funktion der dynamischen Generierung von Event-Notifications verwendet. Diese Methode ermöglicht es, bei einer späteren Erweiterung der Logquellen die Infrastruktur durch einfaches Ergänzen eines Elements in einer Liste flexibel anzupassen, wodurch ein möglichst geringer zusätzlicher Aufwand entsteht.

Listing 1: Dynamische Generierung von SNS-Topics für S3-Bucket-Notifications

```
resource "aws_s3_bucket_notification" "s3_notification" {
    bucket = local.bucket_name
    dynamic "topic" {
        for_each = toset(local.queue_list)
        content {
            events        = ["s3:ObjectCreated:*"]
            filter_prefix = "${topic.key}/"
            id            = "cdc-tsec-siem-${topic.key}-topic"
            topic_arn     = aws_sns_topic.sns_topic[topic.key].arn
        }
    }
}
```

Ebenso wie die Bucket-Notification werden auch die SNS-Topics und die SQS-Queues mithilfe der gleichen Liste dynamisch erstellt. Somit kann sichergestellt werden, dass in jedem Fall die entsprechende Topic für die Benachrichtigung vorhanden ist. Aber auch die entsprechende SQS-Queue ist durch das automatische Generieren definitiv vorhanden und kann mit der Topic verknüpft werden. Die einzelnen Elemente der Liste sind in dieser Arbeit gleichzeitig auch der Beginn der Pfade für die Event-Notification.

Die Definition der SNS-Topics sowie die zugehörigen Policies sind im Anhang C dokumentiert. Der vollständige Terraform-Code zur Erstellung der SQS-Queues ist im Anhang B aufgeführt.

Abschließend erfolgt die Verknüpfung der SQS-Queues mit den entsprechenden SNS-Topics, wie ebenfalls im Anhang C dargestellt. Da sowohl die Queue als auch die Topic im selben AWS-Account angelegt werden, wird die Verbindung automatisch bestätigt und ermöglicht eine reibungslose Integration

5 Architektonische Herausforderungen

5.1 Herausforderungen in der bestehenden Architektur

Fehlende Infrastructure as Code (IaC)

Die aktuelle SIEM-Infrastruktur weist strukturelle Defizite hinsichtlich der Nachvollziehbarkeit und Reproduzierbarkeit auf, da essentielle Ressourcen bislang manuell konfiguriert wurden. Diese Vorgehensweise erhöht einerseits das Fehlerrisiko und führt andererseits zu inkonsistentem Ressourcen-Management. Die Einbindung entscheidender Komponenten in Terraform ist hierbei von besonderer Bedeutung.

Komplexität durch parallelen Betrieb

Der parallele Betrieb zweier SIEM-Systeme erhöht die Komplexität der Architektur. Die erhöhte Komplexität ergibt sich aus dem Limit der S3-Notifications, die jeweils nur eine Benachrichtigung pro Prefix zulassen. Aber auch die Betriebskosten erhöhen sich, durch die anfallenden Kosten für die SNS-Topic, genau so wie die doppelten Kosten für die SQS-Queues. Zusätzlich steigt die Latenz der übertragenden Nachrichten um Millisekunden bis hin zu wenigen Sekunden, weshalb die steigende Latenz vernachlässigbar ist.

Rechte- und Rollenkonzept

Abschließend ist eine präzise Abstimmung aller Berechtigungen und Rollen essentiell, um unautorisierten Zugriff und unerwünschte Aktivitäten innerhalb des Systems auszuschließen.

5.2 Fehlerbehandlung und Schutzmaßnahmen

Verwendung von Dead Leatter Queue (DLQ) und deren Notwendigkeit

Eine zentrale Anforderung seitens Splunk an SQS-Queues besteht darin, dass diesen eine sogenannte Dead Letter Queue (DLQ) zugeordnet wird. Die Funktion einer DLQ liegt darin, Nachrichten zu sammeln, welche von der primären SQS-Queue entweder nicht verarbeitet werden konnten oder über einen längeren Zeitraum hinweg unbeachtet blieben. Dies ermöglicht eine gezielte Analyse und Nachverfolgung von Fehlern im System, wobei unerkannte oder wiederholt problematische Nachrichten identifiziert werden können. Wenn eine Nachricht in die Dead Letter Queue gelangt, bleibt diese bis zu vier Tage in dieser, bis sie entgültig gelöscht wird, oder schon vorher gelöscht wird.

Die Verwendung einer DLQ entspricht bewährten Methoden (Best Practices) und ist sowohl in den unternehmensinternen Datenschutzvorgaben als auch in den Konzernrichtlinien verbindlich vorgeschrieben. Daher ist der Einsatz einer DLQ zwingend erforderlich.

Implementierung von CloudWatch-Alarmen

Ergänzend hierzu verlangen unternehmensinterne Regularien die Einrichtung von CloudWatch-Alarmen für jede DLQ. Konkret müssen zwei Alarme konfiguriert werden:

- Alarm 1: Dieser Alarm wird ausgelöst, sobald die älteste sichtbare Nachricht in der DLQ ein Alter von mindestens 60 Sekunden erreicht. Als sichtbar gelten hierbei Nachrichten, die abgerufen werden können und momentan von keinem anderen Prozess verarbeitet

werden. Die Metrik wird in einminütigen Intervallen über 30 aufeinanderfolgende Perioden überwacht, um den Vorgaben bezüglich Reaktionszeit und Zuverlässigkeit zu entsprechen.

- Alarm 2: Ein zweiter Alarm dient als Indikator für Rückstau. Dieser wird ausgelöst, wenn sich mindestens 100 sichtbare Nachrichten über einen Zeitraum von 30 Minuten in der DLQ befinden. Dies deutet darauf hin, dass eine erhebliche Anzahl von Nachrichten bislang nicht erfolgreich verarbeitet werden konnte.

Verschlüsselung und Schlüsselmanagement

Ein weiteres zentrales Anliegen betrifft die Verschlüsselung sensibler Ressourcen wie S3-Bucket, SNS-Topics sowie SQS-Queues. Zu Beginn war der betreffende S3-Bucket bereits mit einem KMS-Schlüssel versehen, welcher so angepasst werden musste, dass das vorhandene SIEM-System darauf zugreifen konnte. Zusätzlich wurde die Key Policy erweitert, um auch dem Splunk SIEM die Entschlüsselung zu ermöglichen. Für neu erstellte Ressourcen, wie zusätzliche SQS-Queues und SNS-Topics, wurde direkt die Nutzung eines neuen KMS-Schlüssels implementiert, um eine Einbindung in Infrastructure as Code (IaC)-Lösungen sicherzustellen. Auch diese Herangehensweise ist als Best Practice zu verstehen und in unternehmensinternen Vorgaben festgehalten.

5.3 Problematik bei S3-Notifications und Lösung mittels SNS

Im Zuge der Implementierung zeigte sich ein Problem bei der Einrichtung mehrerer S3-Bucket-Notifications auf denselben Pfad. Da beide, das bestehende und das neue SIEM-System parallel mit identischen Daten versorgt werden sollen, musste eine technische Lösung gefunden werden. AWS erlaubt keine doppelte Event-Zuweisung für eine Kombination aus einem Prefix, Suffix und Event. Für die Realisierung wurde daher eine SNS-Topic eingeführt, über welche S3-Events empfangen und auf die jeweils abonnierten SQS-Queues verteilt werden. Dadurch gelingt es, beide SIEM-Systeme parallel mit identischen Daten zu beliefern, ohne potenzielle Risiken oder Informationsverluste einzugehen. Nach der vollständigen Migration kann die SNS-Topic entfernt und die Splunk-SQS-Queue direkt an die S3-Bucket-Notification angebunden werden, ohne dass die bestehende Funktionalität beeinträchtigt wird.

6 Fazit und Ausblick

Das Ziel dieser Arbeit bestand darin zu analysieren, wie Splunk als SIEM-System in einer AWS-Umgebung implementiert und dabei eine unterbrechungsfreie Migration ohne Datenverlust realisiert werden kann. Im Mittelpunkt stand dabei die detaillierte Untersuchung und Planung der notwendigen Architektur, um sowohl den Arbeitsaufwand einzuschätzen als auch spezifische Umsetzungsschritte systematisch festzulegen. Besonders deutlich wurde, dass eine sorgfältige Planung essenziell ist – vor allem im Hinblick auf AWS-spezifische Limitierungen. Beispielhaft wurde dies am Umgang mit S3-Notifications sichtbar: Da S3 keine mehrfachen identischen Event-Zuweisungen zulässt, ist ein SNS-basiertes Fan-out-Verfahren unerlässlich. Zudem zeigte sich, dass die Pull-Ingestion über eine SQS-Queue dem Push-Ansatz überlegen ist, da sie eine zuverlässigere und sichere Datenübertragung ermöglicht.

Durch den Einsatz von Terraform als Infrastructure-as-Code konnte die Nachvollziehbarkeit und langfristige Wartbarkeit der gesamten Lösung erheblich gesteigert werden. Die vollständige Abbildung der Infrastruktur als Code erwies sich daher als grundlegender Baustein für eine robuste sowie zukunftssichere Architektur. Im Ergebnis gewährleistet die entwickelte Lösung eine verlustfreie Eventweiterleitung an beide SIEM-Systeme, die unabhängig voneinander operieren, sodass es nicht zu gegenseitigen Beeinträchtigungen kommt. Die Implementierung von Dead Letter Queues (DLQs) und CloudWatch-Alarmen erhöht zusätzlich die Fehlertoleranz und Überwachungssicherheit. Zudem werden alle Nachrichten und Kommunikationswege durch KMS-gestützte Verschlüsselung geschützt und damit die Vorgaben hinsichtlich Datenschutz und Compliance für S3-Buckets, SNS-Topics und SQS-Queues gleichermaßen erfüllt.

Das dynamische Design des Codes fördert nicht nur die Erweiterbarkeit, sondern erleichtert auch die Lesbarkeit und das Verständnis der Infrastruktur. Gleichwohl stellen Migrationen im Sicherheitsbereich hohe Anforderungen an gründliche Risikoanalysen und den Schutz sensibler Daten. Essenziell bleibt, dass das SIEM-System kontinuierlich Zugang zu den benötigten Daten für eine zeitnahe Risikoanalyse behält, um im Ernstfall rasch eingreifen zu können. Der temporäre Parallelbetrieb beider SIEM-Systeme ist zwar mit erhöhten Kosten und einer komplexeren Infrastruktur verbunden, bleibt jedoch für eine qualitativ hochwertige Migration unverzichtbar.

Zukünftig sollten Optimierungen hinsichtlich der Datenqualität und Log-Hygiene vorgenommen werden, da die Leistungsfähigkeit des SIEM-Systems maßgeblich von der Qualität der eingehenden Logdaten abhängt. Mögliche Weiterentwicklungen beinhalten die Integration zusätzlicher Logquellen wie VPC Flow Logs, um die Transparenz des Netzwerkverkehrs weiter zu erhöhen, sowie das Einbinden von AWS Config, um Konfigurationsänderungen zentral auszuwerten und potenzielle Sicherheitsrisiken schon frühzeitig zu erkennen.

Abschließend bleibt festzuhalten, dass nach Abschluss der Migration der Rückbau der Infrastruktur für den Parallelbetrieb wichtig ist. Dazu zählen insbesondere die Stilllegung nicht mehr benötigter SQS-Queues und SNS-Topics, sobald ausschließlich das neue SIEM-System in Betrieb ist und pro Event jeweils nur noch eine SQS-Queue benachrichtigt werden muss.

Literaturverzeichnis

Anthony, P. et al. (o.J.). Systematic Review of SIEM Integration for Threat Detection and Log Correlation in AWS-Based Infrastructure. In.

Basatwar, G. (o.J.). Splunk SIEM - Meaning, Features & Importance| Doverunner. Section: Blogs. in: https://doverunner.com/blogs/splunk-siem-guide/, (09. 02. 2026).

Clouidan (o.J.). Splunk Architecture: Components and Best Practices. in: https://cloudian.com/guides/splunk-big-data/splunk-architecture-data-flow-components-and-topologies/, (09. 02. 2026).

Logging and Monitoring - AWS Network Firewall (o.J.). in: https://docs.aws.amazon.com/network-firewall/latest/developerguide/proxy-logging-and-monitoring.html, (21. 02. 2026).

Ojuko, D. A. (o.J.). Splunk SIEM Tool in Cybersecurity: A Comprehensive Overview. in: https://www.linkedin.com/pulse/splunk-siem-tool-cybersecurity-comprehensive-overview-dada-ojuko-bjrxc, (09. 02. 2026).

SEC04-BP02 Erfassen von Protokollen, Erkenntnissen und Metriken an standardisierten Orten - AWS Well-Architected Framework (o.J.). in: https://docs.aws.amazon.com/de_de/wellarchitected/latest/framework/sec_detect_investigate_events_logs (21. 02. 2026).

Sheeraz, M. et al. (Mai 2023). Effective Security Monitoring Using Efficient SIEM Architecture. In: *Human-centric Computing and Information Sciences* 13. DOI: 10.22967/HCIS.2023.13.023.

Splunk (o.J.). Splunk Enterprise Security | Splunk. in: https://www.splunk.com/en_us/products/enterprise-security.html, (09. 02. 2026).

Vielberth, M. und Pernul, G. (2018). A security information and event management pattern. In.

Anhang

Anhangsverzeichnis

Anhang A IAM-Rolle und Policies für Splunk SIEM 20

Anhang B SQS-Queue Definition und SQS-Policy 22

Anhang C SNS-Topic Definition und SNS-Policy 25

Anhang A: IAM-Rolle und Policies für Splunk SIEM

Listing 2: IAM-Rolle und zugehörige Policies für Splunk SIEM

```
# Create IAM Role for Splunk SIEM to access SQS and S3
resource "aws_iam_role" "splunk_role" {
  name = "splunk_siem_role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect = "Allow",
        Principal = {
          AWS = ["arn:aws:iam::${data.aws_caller_identity.current.id}:r
        },
        Condition = {
          StringEquals = {
            "AWS:PrincipalArn" = [aws_iam_user.user.arn]
          },
        },
        Action = "sts:AssumeRole"
      }
    ]
  })
}
# Create IAM Policies for Splunk SIEM to access all SQS
resource "aws_iam_policy" "splunk_sqs_policy" {
  name        = "splunk_siem_sqs_policy"
  description = "Policy␣for␣Splunk␣SIEM␣to␣access␣SQS"
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Sid    = "SplunkSIEMqueuePolicy",
        Effect = "Allow",
        Action = [
          "sqs:ChangeMessageVisibility",
          "sqs:DeleteMessage",
          "sqs:GetQueueAttributes",
          "sqs:GetQueueUrl",
          "sqs:ReceiveMessage",
          "sqs:SendMessage"
        ],
        Resource = local.queue_arns,
      },
      {
        Sid    = "ListAllSQSQueues",
        Effect = "Allow",
        Action = [
```

```
            "sqs:ListQueues"
          ],
          Resource = "*",
        }
      ]
    })
}
# IAM Policy for Splunk role to get Objects out of the Bucket
resource "aws_iam_policy" "splunk_s3_policy" {
  name        = "splunk_siem_s3_policy"
  description = "Policy for Splunk SIEM to access  S3"
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Sid    = "SplunkSIEMs3Policy",
      Effect = "Allow",
      Action = [
        "s3:GetObject",        # if Versioning is disabled
        "s3:GetObjectVersion", # If Versioning is enabled
        "s3:ListBucket"
      ],
      Resource = local.bucket_policy_resource,
    }]
  })
}
# IAM Policy for Splunk role to get Access for splunk to both kms keys
resource "aws_iam_policy" "splunk_kms_policy" {
  name        = "splunk_siem_kms_policy"
  description = "Policy for Splunk SIEM to access KMS"
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Sid    = "SplunkSIEMkmsPolicy",
      Effect = "Allow",
      Action = [
        "kms:Decrypt",
        "kms:Encrypt",
        "kms:DescribeKey"
      ],
      Resource = local.kms_role_policy_resource,
    }]
  })
}
```

Anhang B: SQS-Queue Definition und SQS-Policy

```
,
# Create a SQS Queue with KMS encryption and link to a Dead Letter Queu
resource "aws_sqs_queue" "queue" {

  for_each                          = toset(local.queue_list)
  provider                          = aws
  name                              = join("-", ["cdc-tsec-siem", each.
  kms_data_key_reuse_period_seconds = 300
  kms_master_key_id                 = aws_kms_alias.kms_splunk_alias.id
  visibility_timeout_seconds        = 900
  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.dlq[each.key].arn
    maxReceiveCount     = 5
  })
}

# Create a SQS Queue wich is used as a Dead Letter Queue (DLQ)
resource "aws_sqs_queue" "dlq" {
  for_each                          = toset(local.queue_list)
  provider                          = aws
  name                              = join("-", ["cdc-tsec-siem", each.
  kms_data_key_reuse_period_seconds = 300
  kms_master_key_id                 = aws_kms_alias.kms_splunk_alias.id
  visibility_timeout_seconds        = 900

}

# Create a SQS Queue Policy to allow S3 to send messages to the SQS que
resource "aws_sqs_queue_policy" "policy" {
  for_each  = toset(local.queue_list)
  queue_url = aws_sqs_queue.queue[each.key].id
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect    = "Allow",
        Principal = { Service = ["s3.amazonaws.com", "sns.amazonaws.com
        Action = [
          "sqs:GetQueueAttributes",
          "sqs:GetQueueUrl",
          "sqs:ListDeadLetterSourceQueues",
          "sqs:ListQueues",
          "sqs:SendMessage"
        ],
        Resource = aws_sqs_queue.queue[each.key].arn,
        Condition = {
```

```
            ArnEquals = {
              "aws:SourceArn" = aws_sns_topic.sns_topic[each.key].arn
            },
            Bool : {
              "aws:SecureTransport" : "true"
            },
          }
        },
        {
          Effect    = "Allow",
          Principal = { "AWS" : aws_iam_role.splunk_role.arn },
          Action = [
            "sqs:ChangeMessageVisibility",
            "sqs:DeleteMessage",
            "sqs:GetQueueAttributes",
            "sqs:GetQueueUrl",
            "sqs:ListQueues",
            "sqs:ReceiveMessage",
            "sqs:SendMessage"
          ],
          Resource = aws_sqs_queue.queue[each.key].arn,
          Condition = {
            # ArnEquals = {
            #   "aws:SourceArn" = data.aws_s3_bucket.s3.arn
            # },
            Bool : {
              "aws:SecureTransport" : "true"
            },
          }
        }
      ]
    })
}

# Create a SQS Queue Policy to allow S3 to send messages to the SQS DLQ
resource "aws_sqs_queue_policy" "policy_dlq" {
  for_each  = toset(local.queue_list)
  queue_url = aws_sqs_queue.dlq[each.key].id
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect    = "Allow",
        Principal = { Service = ["sqs.amazonaws.com"] },
        Action = [
          "sqs:SendMessage"
        ],
        Resource = aws_sqs_queue.dlq[each.key].arn,
```

```
      Condition = {
        "ArnEquals" = {
          "aws:SourceArn" = aws_sqs_queue.queue[each.key].arn
        },
        "Bool" : {
          "aws:SecureTransport" : "true"
        },
      }
    },
    {
      Effect    = "Allow",
      Principal = { "AWS" : aws_iam_role.splunk_role.arn },
      Action = [
        "sqs:ChangeMessageVisibility",
        "sqs:GetQueueAttributes",
        "sqs:GetQueueUrl",
        "sqs:ListQueues",
        "sqs:ListDeadLetterSourceQueues",
        "sqs:ReceiveMessage"
      ],
      Resource = aws_sqs_queue.dlq[each.key].arn,
      Condition = {
        # ArnEquals = {
        #   "aws:SourceArn" = data.aws_s3_bucket.s3.arn
        # },
        "Bool" : {
          "aws:SecureTransport" : "true"
        },
      }
    }
  ]
 })
}
```

Anhang C: SNS-Topic Definition und SNS-Policy

```
,
resource "aws_sns_topic" "sns_topic" {
  for_each          = toset(local.queue_list)
  name              = join("-", ["cdc-tsec-siem", each.key, "topic"])
  kms_master_key_id = aws_kms_alias.kms_splunk_alias.id
  display_name      = "CDC TSEC SIEM Topic ${each.key}"
}

resource "aws_sns_topic_policy" "sns_topic_policy" {
  for_each = toset(local.queue_list)
  arn      = aws_sns_topic.sns_topic[each.key].arn
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect = "Allow",
        Sid    = "CdcTsecSiem${each.key}SQS",
        Principal = {
          Service = "sqs.amazonaws.com"
        },
        Action = [
          "sns:Subscribe",
          "sns:Receive"
        ],
        Resource = aws_sns_topic.sns_topic[each.key].arn
      },
      {
        Effect = "Allow",
        Sid    = "CdcTsecSiem${each.key}S3",
        Principal = {
          Service = "s3.amazonaws.com"
        },
        Action = [
          "sns:Publish"
        ],
        Resource = aws_sns_topic.sns_topic[each.key].arn
      }
    ]
  })
}

resource "aws_sns_topic_subscription" "sns_topic_subscription_queue_lis
  for_each             = toset(local.queue_list)
  topic_arn            = aws_sns_topic.sns_topic[each.key].arn
  protocol             = "sqs"
  raw_message_delivery = true
```

```
  endpoint                = aws_sqs_queue.queue[each.key].arn
}
```

Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Arbeit selbständig und ohne unerlaubte fremde Hilfe angefertigt, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Stellen sind als solche kenntlich gemacht. Die Zustimmung des Partnerunternehmens in der Praxis zur Verwendung betrieblicher Unterlagen habe ich eingeholt. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Dresden, 27.02.2026

Ort, Datum Unterschrift des Verfassers